Part B: Modelling

Preprocessing, feature engineering and two-model implementation

Author	Vusumuzi Nkosi
Student number	26303296
Module	CAP182 Capstone Project
Document status	Word review report — no final PDF created

Report contents

- Preprocessing and leakage control
- Feature engineering
- Model 1: Logistic Regression
- Model 2: XGBoost

Preprocessing (CAP182 SS2 - Part B)

Related scripts	scripts/common.py and scripts/preprocessing.py

How to run

```
python scripts/preprocessing.py
```

Smoke test reads the public dataset, constructs the fixed training split, fits the preprocessing objects using training data only and prints out the shape of the processed matrix. Preprocessing used in both model pipelines is done automatically.

Data definition and split

The datafile consists of 41,188 entries, 20 input variables and binary target variable y. Target variable is encoded to 1 when 'yes' (subscribed) and 0 when 'no'. The stratified 80/20 split with random seed 42 results in 32,950 rows in training set and 8,238 rows in test set with 11.3% positive rate. Same number of rows is used for both models.

Duration is dropped before any split passes down to a downstream transformer. Duration is known only after the marketing call has been completed and is not known at the scoring time. Dropping it avoids target leakage and complies with the SS1 rule, according to which STADIOEquities model can use only available information at its scoring cut-off.

Input columns after leakage control

Type	Count	Columns
Numeric	9	age, campaign, pdays, previous, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed
Categorical	10	job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome

Two numeric features and two categorical ranges are included through feature engineering in the next step. The transformed preprocessor will yield 73 model input features from the 23 engineered features.

Processing pipeline

build_preprocessor() returns a scikit-learn ColumnTransformer with the following branches:

Branch	Steps	Purpose
Numeric	Median imputation, then StandardScaler	Offers an excellent alternative for the missing pdays after cleaning and equalizes the scales of numeric variables.
Categorical	Most-frequent imputation, then one-hot encoding	Transforms categories to numeric indicators while ignoring unknown categories encountered during inference.

Selection of the features is done based on their data types; hence, the engineered features will pass through without an externally managed list of features. Standardisation is necessary for Logistic Regression since the estimation of coefficients and L2 regularisation is sensitive to scale. It is unnecessary for tree-based splits; however, applying the same fitted preprocessor ensures fairness of the comparison.

Leakage and reproducibility controls

The feature engineer, imputers, scaler, and encoder are fitted only in the scikit-learn Pipelines in the training folds. The test data is transformed with these fitted objects, such that test medians, category frequencies, and scaling statistics do not get into the training fold. The random seed, test size, target mapping, and excluded features are centralized in scripts/common.py to ensure that the two models do not diverge in any way.

The official CSV used for the verified run has SHA-256:

74adfc578bf77a7ff4bb1ba4a9f8709d9e3c6907342959c2c8416847e0afb4d8

Feature Engineering (CAP182 SS2 - Part B)

Related script	scripts/feature_engineering.py

How to run

```
python scripts/feature_engineering.py
```

The script gives a preview of the engineered features in the fixed training fold. The feature engineer is the first step in both of the models, and the process is done similarly.

Engineered features

Transformation	Definition	Modelling rationale	STADIOEquities analogue
was_prev_contacted	1 when pdays is not the 999 sentinel; otherwise 0	Separates the prior existence of a campaign relationship from the actual number of days since last contact.	Whether a new account previously had a meaningful engagement with the service.
pdays (cleaned)	Replace the 999 sentinel with a missing value	Makes sure that a “never contacted” code does not turn into an unrealistically large number of elapsed days; the numeric imputer handles it downstream.	Avoid modeling “no prior event” as a separate value from actual time-since-event values.
has_prior_contacts	1 when previous is greater than zero	Detects if there is any relationship history in a simple, understandable way.	Prior user interaction with the app or support team prior to scoring cut-off date.
age_group	Bin age into <=25, 26-35, 36-45, 46-55, 56-65 and 65+	Enables the linear model to account for non-linear life stage effects without assuming a linear effect.	An example of a more general case of turning continuous customer attributes into behavioral bands.
contact_intensity	Bin campaign contacts into 1, 2-3 and 4+	Modeling diminishing marginal utility and possible fatigue from campaign interactions.	Early session or notification intensity bands prior to funding.

Design and leakage control

These transformations are deterministic and stateless, meaning they do not compute means, target rates or class frequencies based on all observations in the data. As a result, the transformations can be safely applied prior to applying the fitted pre-processor, thereby avoiding the use of any test set data. The function transformer used for all four transformations is serialized with the model and included in the pipeline.

The chosen feature set is intentionally compact and interpretable. The feature set allows Logistic Regression some amount of non-linearity while still allowing XGBoost to discover further interactions. Speculation and inclusion of many extra features would complicate the interpretation of the public proof-of-viability and potentially cause overfitting in one particular train/test split.

Limitations and next step

The public data relates to bank telemarketing, not app activation, which is why the chosen features set up some basic modeling patterns but are not intended as a final production feature set. Once the STADIOEquities data becomes available, the same principles will have to be applied to modeling KYC completion, early app activity, source of acquisition, support events and elapsed-time features up until the agreed 24-hour scoring cut-off.

Model 1 - Logistic Regression (CAP182 SS2 - Part B)

Related scripts	scripts/train_model1_logreg.py, scripts/evaluate.py and scripts/explain_models.py

How to run

```
python scripts/train_model1_logreg.py
python scripts/explain_models.py
```

The script for training the model is part of the full pipeline, makes inference on the held-out test set, prints the metrics and saves the results and the fitted pipeline. The interpretation script runs after both models have been fitted.

Technique and role

Logistic Regression models the logarithm of the odds of the positive class as a weighted sum of the inputs. It is our baseline for interpretability, because it is quick to train, well-known, and gives coefficients that are indicative of whether fitted association is increasing or decreasing the subscription score. Having such a good baseline model makes sure that a more complex model does not get recommended just because it is fancier.

Pipeline

feature engineering -> preprocessing -> LogisticRegression

The whole scikit-learn Pipeline is fitted only using the training fold and saved as models/model1_logreg.joblib. Because the transformations are stored within the classifier, the same feature transformation is applied during inference.

Hyperparameters

Parameter	Value	Rationale
solver	lbfgs	Stable optimizer for L2-regularized binary logistic regression.
max_iter	1000	Ensures adequate iterations to converge once data is one-hot encoded.
class_weight	balanced	Gives higher weights to the minority class depending on how it is distributed during training, rather than duplicating instances.
penalty	L2 (default)	Penalizes features and lowers variance if the encoded predictors are correlated.
C	1.0 (default)	Gives an easily interpretable unoptimized model for comparing against Model 2.
random_state	42	Reproducibility.
decision threshold	0.50	Ensured consistent common operating point for the head-to-head comparison.

Fitted associations

Top positive fitted coefficients are: March contacts, Consumer Price Index, Cellular contact, and Pre-contact status. Top negative coefficients are: Employment variation, some Campaign months, Telephone contact and Contact count in Campaign. All of these are only conditionally associated, not causal factors; the use of one-hot categorization and of macroeconomic variables introduces correlation such that individual coefficient values need to be read carefully.

The entire ranked output can be found at experimental-results/feature_effects.md. Its purpose is to document the baselines for auditability, not to imply that varying any single factor will produce a subscription.

Outputs

Output	Purpose
experimental-results/model1_metrics.json	Reproducible held-out metrics and confusion-matrix counts.
experimental-results/model1_predictions.csv	Aligned row-wise true labels, probabilities and predictions for comparisons.
models/model1_logreg.joblib	Serialisation of entire train-to-prediction pipeline (excluding from Git as it is replicable).

Model 2 - XGBoost (CAP182 SS2 - Part B)

Related scripts	scripts/train_model2_xgboost.py, scripts/evaluate.py and scripts/explain_models.py

How to run

```
python scripts/train_model2_xgboost.py
python scripts/explain_models.py
```

The pipeline script trains on the entire pipeline, does inference on the held out test rows for Model 1, prints metrics and stores the results and trained model.

Technique and role

XGBoost is a gradient-boosted decision tree algorithm. It grows decision trees in an incremental way, where each subsequent decision tree addresses the residual errors from the previous ensemble. In contrast to the baseline model, it learns non-linear boundaries and interactions between the variables without being explicitly programmed beforehand. It represents a stronger choice of the candidate model, given the trade-off with greater complexity and lower transparency.

Pipeline

feature engineering -> preprocessing -> XGBClassifier

The trained pipeline is stored under the name models/model2_xgboost.joblib.

Hyperparameters

Parameter	Value	Rationale
n_estimators	400	Sufficient number of boosting iterations in combination with a conservative learning rate.
learning_rate	0.05	Smaller steps increase stability and lower the chance of overfitting individual trees.
max_depth	5	Covers moderately interacting features without increasing tree complexity.
subsample	0.8	Random sampling of rows per tree for regularization.
colsample_bytree	0.8	Random sampling of features per tree for regularization.
reg_lambda	1.0	Uses L2 regularization for leaf weights.
min_child_weight	1	Keeps the usual minimum child weight constraint.
scale_pos_weight	7.88	Maintains the proportion of negative-to-positive observations from the training split (29,238 / 3,712).
tree_method	hist	Fast tree growth using histograms.
objective / metric	binary:logistic / logloss	Outputs probability scores in binary classification problems.
random_state	42	Reproducibility.
decision threshold	0.50	Standard operating point for the given comparison.

They were chosen beforehand, prior to the held-out final evaluation. Cross-validation is advised beforehand before going into production, but the test dataset should not be tainted by it.

Fitted importance summary

The most important features in XGBoost are nr.employed, October contact, was_prev_contacted, emp.var.rate, cons.conf.idx and success from previous campaign. The importance of a feature is not directional; it means that the learned trees use a feature to minimize loss, rather than the feature increasing the score or causing the effect itself.

Outputs

Output	Purpose
experimental-results/model2_metrics.json	Reproducible held-out metrics and confusion matrix tallies.
experimental-results/model2_predictions.csv	Row-aligned true labels, probabilities, and predictions for comparison purposes.
models/model2_xgboost.joblib	Serialized training-to-prediction workflow (omitted from Git because it can be reproduced).